

Impacto de los Parámetros Libres en el Aprendizaje de una Red Neuronal

Geometría del Espacio de Solución Teórico y Dinámica de Optimización

Análisis Teórico y Experimental

Red Neuronal de Demanda (`red_neuronal.py`)

5 de octubre de 2026

Deep Learning & Optimización Matemática

Mapa de Parámetros Libres en el Código

En `red_neuronal.py`, el entrenamiento está gobernado por hiperparámetros que definen el espacio de hipótesis $\mathcal{H}$ y la trayectoria de optimización:

Parámetro	Valor en Código	Rol Matemático	Efecto Principal en el Espacio
Tasa Aprendizaje (η)	0.001	Escala del paso de gradiente	Velocidad vs. oscilación en la cuenca
Escalado de Datos	<code>StandardScaler</code>	Condicionamiento del Hessiano	Esferificación de contornos elípticos
Tamaño de Lote (B)	64	Varianza del gradiente estocástico	Balance ruido térmico vs. aceleración vectorial
Número de Épocas (E)	100	Pasos sobre el dataset completo	Trade-off Sesgo-Varianza / Sobreajuste
Arquitectura	$5 \rightarrow 16 \rightarrow 8 \rightarrow 1$	Capacidad del espacio $\mathcal{H}$	Expresividad sin memorización espuria
Activación	<code>ReLU</code>	Operador de no-linealidad	Plegado del espacio latente sin desvanecimiento
Optimizador	<code>Adam</code>	Momentos cinemáticos adaptativos	Amortiguación y ajuste dimensional de paso
Función Pérdida	<code>MSELoss</code>	Métrica euclídea cuadrática	Convexidad local y sensibilidad a anomalías

Definición Formal de la Optimización

Dado el conjunto de datos $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, buscamos el vector óptimo $\theta^* \in \mathbb{R}^D$ ($D = 241$ pesos en nuestra red):

$$\theta^* = \arg \min_{\theta \in \Theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f(x_i; \theta), y_i)$$

Aproximación Local de Taylor (2º Orden)

Alrededor de un punto estacionario θ^* :

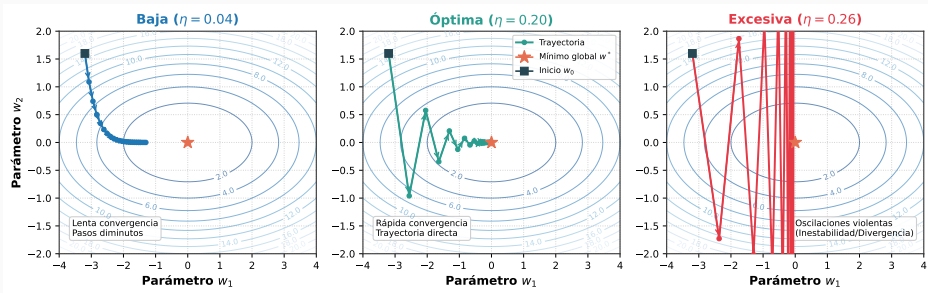
$$J(\theta) \approx J(\theta^*) + \frac{1}{2}(\theta - \theta^*)^T H(\theta - \theta^*)$$

donde $H = \nabla^2 J(\theta^*)$ es la **matriz Hessiana** de curvatura.

Geometría y Cinemática del Aprendizaje

- Los **autovalores de H** ($\lambda_{\max}, \lambda_{\min}$) definen la rigidez y elongación de la cuenca.
- Si $\lambda_{\max}/\lambda_{\min} \gg 1$, el paisaje tiene **gargantas estrechas y valles planos**.
- La elección de hiperparámetros modula:
 1. La **geometría del paisaje**: preprocesamiento, regularización y función de pérdida.
 2. La **cinemática de búsqueda**: tasa η , tamaño de lote B y optimizador.

Parámetro 1: Tasa de Aprendizaje (η / $1r$)



Tasa Baja ($\eta = 0.04$)

- Pasos diminutos en el gradiente.
- Convergencia lenta; riesgo de estancamiento en mesetas o mínimos locales planos.

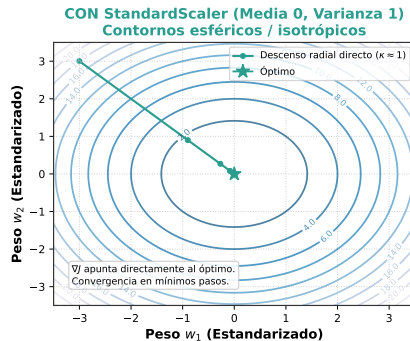
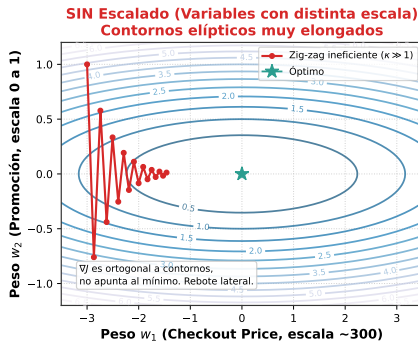
Tasa Óptima ($\eta = 0.20$)

- Descenso eficiente hacia el óptimo θ^* .
- Teóricamente $\eta_{\text{opt}} = \frac{2}{\lambda_{\text{máx}} + \lambda_{\text{mín}}}$ en cuencas parabólicas.

Tasa Excesiva ($\eta = 0.26$)

- “Overshooting” y rebote entre paredes opuestas de la cuenca.
- Inestabilidad numérica, oscilaciones y divergencia a ∞ o NaN.

Parámetro 2: Escalado de Datos y Condicionamiento del Hessiano



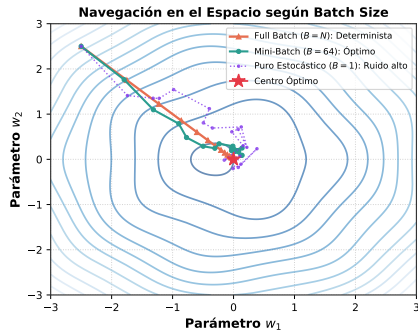
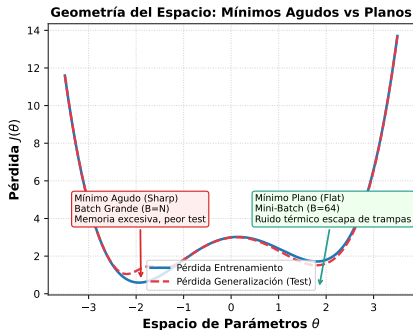
¿Por qué ocurre el Zig-Zag sin Escalar?

- $\text{checkout_price} \sim 300$, mientras $\text{emailer} \in \{0, 1\}$.
- Número de condición $\kappa(H) = \frac{\lambda_{\max}}{\lambda_{\min}} \gg 1$.
- El vector gradiente ∇J es ortogonal a las curvas de nivel, apuntando casi perpendicular al mínimo; genera rebotes laterales inútiles.

Efecto de StandardScaler ($z = \frac{x - \mu}{\sigma}$)

- Lleva la varianza de cada variable a 1.0 y la media a 0.
- Esferifica las curvas de nivel ($\kappa(H) \approx 1$).
- El gradiente apunta **directamente al mínimo teórico**, permitiendo pasos amplios y acelerando drásticamente el aprendizaje.

Parámetro 3: Tamaño de Lote (B) — Ruido y Generalización



Lote Unitario ($B = 1$, SGD puro)

- Varianza máxima del gradiente: $\text{Var}(g_B) \propto 1/B$.
- Caminata errática; ineficiente en hardware moderno por falta de vectorización.

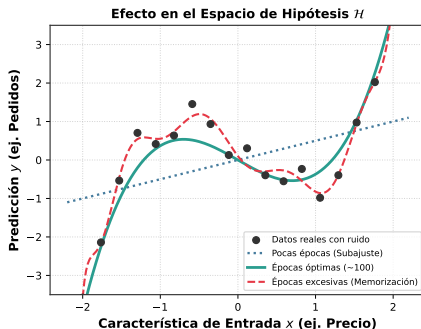
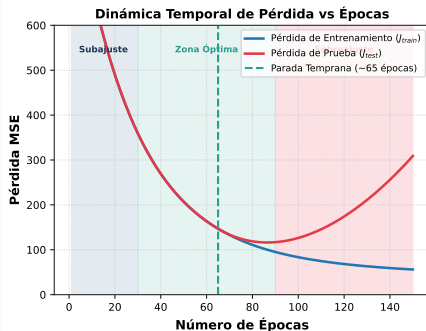
Mini-Batch ($B = 64$, Punto Dulce)

- Ruido térmico óptimo que permite escapar de mínimos agudos (sharp minima).
- Converge a mínimos anchos (flat minima) que ofrecen alta generalización.

Batch Completo ($B = N$)

- Gradiente determinista exacto.
- Alto consumo de memoria; tiende a atraparse en mínimos locales estrechos y sobreajustar.

Parámetro 4: Número de Épocas (E) — Dinámica Temporal



Pocas Épocas ($E < 30$): Subajuste

- Alto sesgo (Underfitting).
- La red no ha alcanzado la cuenca de convergencia; predicciones planas o insuficientes.

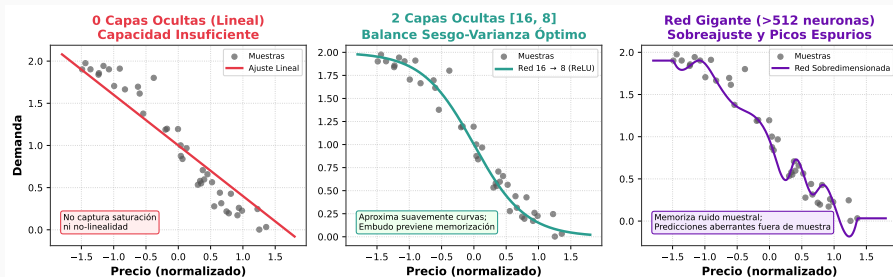
Épocas Óptimas ($E \approx 100$)

- Punto de equilibrio donde el error de prueba J_{test} alcanza su mínimo.
- Captura la señal real subyacente sin memorizar el ruido muestral.

Épocas Excesivas ($E > 150$): Sobreajuste

- Alta varianza (Overfitting).
- $J_{train} \rightarrow 0$ mientras J_{test} diverge.
- La red ajusta oscilaciones espurias interpolando ruido de la muestra.

Parámetro 5: Arquitectura — Capacidad del Espacio de Hipótesis



0 Capas Ocultas (Lineal)

- $\hat{y} = Wx + b$.
- Espacio rígido; no modela la interacción no lineal precio-promoción ni la saturación de demanda.

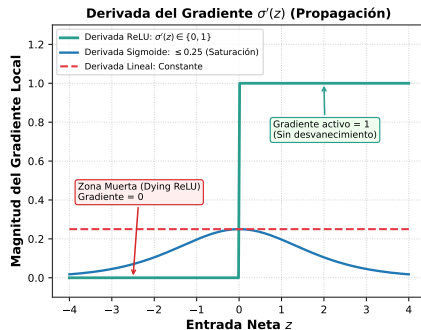
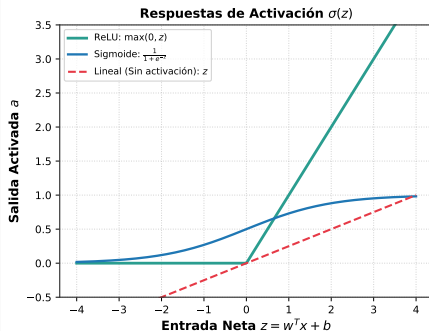
2 Capas [16 → 8] (Embudo)

- $5 \rightarrow 16 \rightarrow 8 \rightarrow 1$ (241 pesos).
- El embudo comprime representaciones abstractas; previene memorización y aproxima curvas suaves.

Red Sobredimensionada (> 512 n)

- Miles de parámetros vs. 1600 datos.
- Espacio de hipótesis gigante: mínimos espurios y predicciones aberrantes fuera de muestra.

Parámetro 6: Función de Activación — No Linealidad y Gradientes



Lineal: Colapso Matemático

- $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + b'$.
- Apilar capas lineales colapsa a una única matriz afín; anula la profundidad.

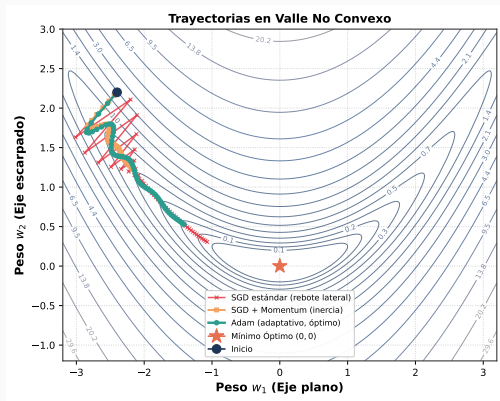
Sigmoid / Tanh: Saturación

- $\sigma'(z) \leq 0.25$.
- Por regla de la cadena: $\prod_{l=1}^L \sigma' \rightarrow 0$.
- Desvanecimiento del gradiente en capas iniciales (Vanishing Gradient).

ReLU: Gradiente Robusto

- $\sigma'(z) = 1$ para $z > 0$.
- Flujo de gradiente limpio sin atenuación; cálculo algebraico ultra veloz.
- *Salida sin activación*: Codominio libre $\mathbb{R}$ para regresión continua.

Parámetro 7: Optimizador — Cinemática Adaptativa en el Paisaje



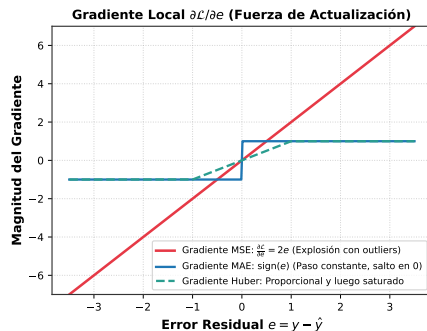
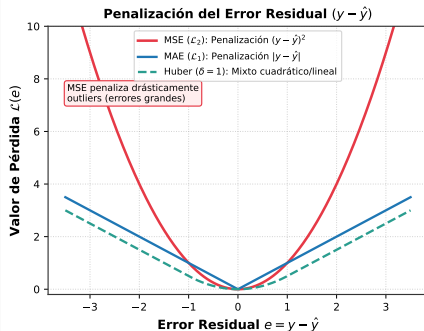
Mecánica de Adam (Adaptive Moments)

- $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ (1º Momento / Inercia)
- $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ (2º Momento / Curvatura)
- $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$

Dinámica en Gargantas Estrechas y Valles

- **SGD estándar:** Sufre en paisajes anisótipos. Da pasos gigantescos en la dirección escarpada y pasos minúsculos en la dirección longitudinal plana.
- **SGD + Momentum:** Acumula inercia en la dirección persistente y cancela oscilaciones transversales.
- **Adam (red_neuronal.py):** Normaliza el paso por parámetro dividiendo por $\sqrt{v_t}$. Avanza con paso firme y rápido en direcciones planas sin desestabilizarse.

Parámetro 8: Función de Pérdida — Geometría del Error y Robustez



Error Cuadrático Medio (MSELoss, L_2)

- $\mathcal{L}_{MSE} = (y - \hat{y})^2 \implies \nabla = 2(y - \hat{y})$.
- Cuenca parabólica suave: gradiente proporcional al error; garantiza convergencia precisa y estable cerca del mínimo.
- *Riesgo*: Errores atípicos (outliers de demanda extrema) generan gradientes muy grandes que pueden perturbar el aprendizaje.

Error Absoluto Medio (L1Loss, MAE)

- $\mathcal{L}_{MAE} = |y - \hat{y}| \implies \nabla = \text{sign}(y - \hat{y})$.
- Gradiente constante e inmune a valores extremos (robusto a outliers).
- *Desventaja*: Discontinuidad del gradiente en 0; tiende a oscilar alrededor del mínimo si la tasa η no decae asintóticamente.

Interacción y Reglas de Co-diseño entre Hiperparámetros

Los parámetros libres no operan de forma aislada; interactúan en el espacio de solución:

Si modificas...	Efecto Secundario	Ajuste Compensatorio Recomendado
Aumentar Batch Size (B)	Reduce el ruido de gradiente	Aumentar η proporcionalmente (Regla de escala lineal $\eta \propto B$).
Aumentar Capas / Ancho	Expande $\mathcal{H}$ y riesgo de sobreajuste	Reducir Épocas o agregar regularización (L_2 weight_decay o Dropout).
Cambiar de Adam a SGD	Pierde adaptabilidad de curvatura	Añadir Momentum ($\beta = 0.9$) y usar un planificador de tasa (Scheduler).
Eliminar StandardScaler	Elipses de pérdida $\kappa(H) \gg 1$	Reducir drásticamente η para evitar divergencia, a costa de convergencia lenta.
Aumentar Épocas ($E > 200$)	Memorización asintótica	Implementar Early Stopping monitoreando la pérdida de validación.

Principios del Espacio Teórico

- La **geometría del paisaje** está determinada por el preprocesamiento, la arquitectura y la pérdida.
- La **cinemática de la trayectoria** está controlada por el optimizador, la tasa η y el batch size B .
- El objetivo no es minimizar a ciegas J_{train} , sino encontrar un **mínimo plano generalizable**.

Protocolo de Diagnóstico en `red_neuronal.py`

- **¿Pérdida es NaN o explota?** $\implies$ Disminuir η o verificar escalado de datos.
- **¿Aprende demasiado lento?** $\implies$ Aumentar η o verificar que no haya neuronas muertas (Dying ReLU).
- **¿ R^2 alto en Train pero bajo en Test?** $\implies$ Sobreajuste: reducir épocas o simplificar arquitectura ($16 \rightarrow 8$).
- **¿ R^2 bajo en ambos?** $\implies$ Subajuste: verificar activación y añadir capacidad.

¿Preguntas o Discusión?

Código fuente de referencia: `red_neuronal.py`
Gráficas generadas con Matplotlib (Vector PDF)